

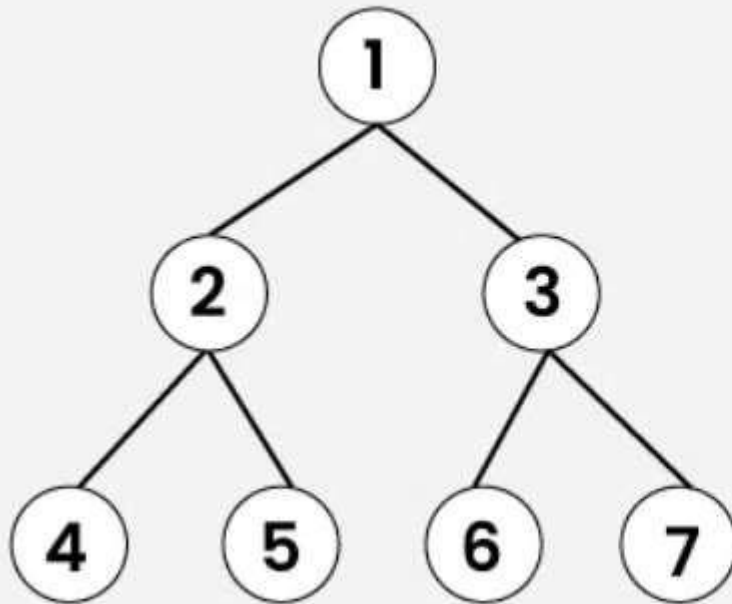
23CS201 - Data Structures and Algorithms using C++

Study Materials

Tree Traversal Techniques:

- Tree Traversal techniques include various ways to visit all the nodes of the tree.
- Unlike linear data structures (Array, Linked List, Queues, Stacks, etc) which have only one logical way to traverse them, trees can be traversed in different ways.
- In next we will discuss all the tree traversal techniques along with their uses.

Tree Traversal:



Inorder Traversal

4	2	5	1	6	3	7
---	---	---	---	---	---	---

Preorder Traversal

1	2	4	5	3	6	7
---	---	---	---	---	---	---

Postorder Traversal

4	5	2	6	7	3	1
---	---	---	---	---	---	---

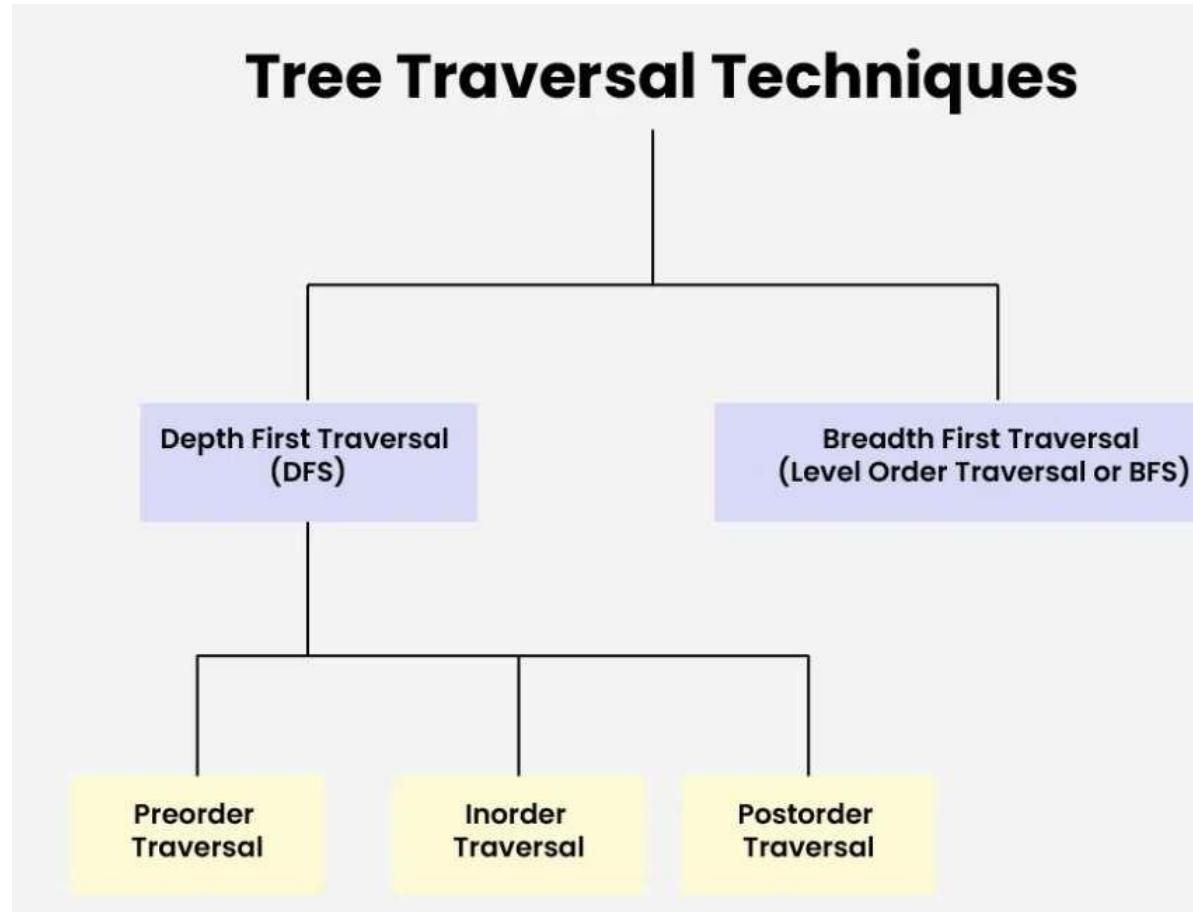
Level order Traversal

1	2	3	4	5	6	7
---	---	---	---	---	---	---

Tree Traversal Techniques:

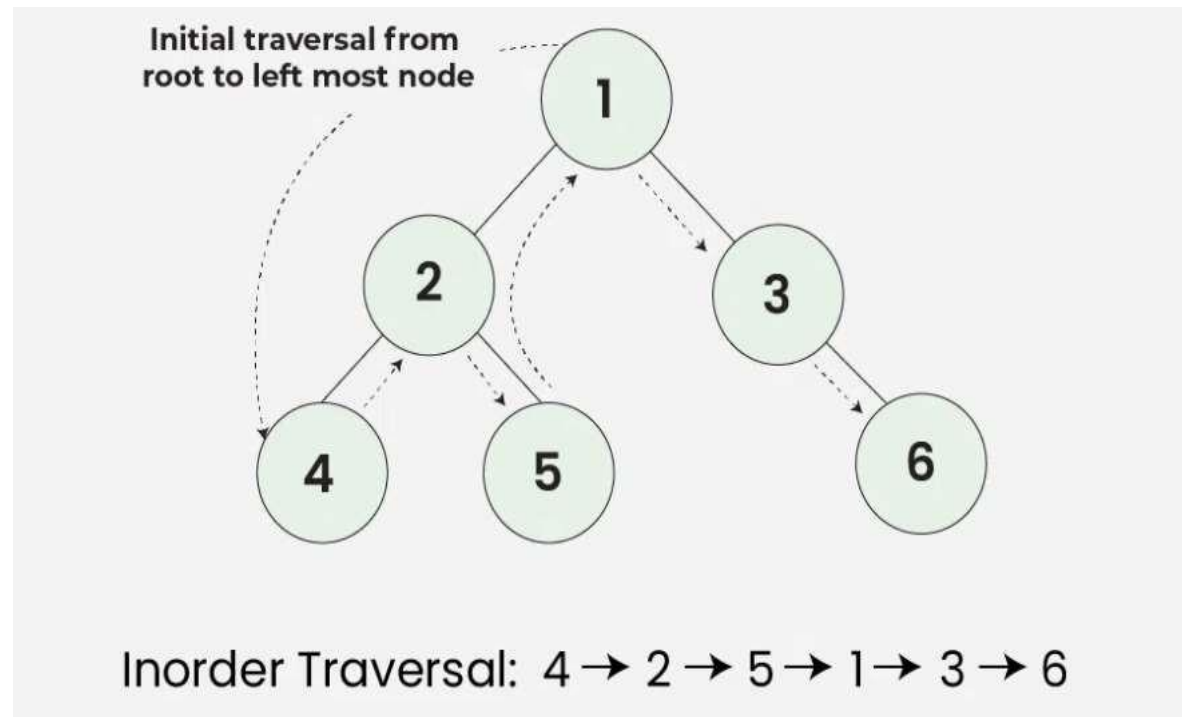
- Tree Traversal refers to the process of visiting or accessing each node of the tree exactly once in a certain order.
- Tree traversal algorithms help us to visit and process all the nodes of the tree.
- Since tree is not a linear data structure, there are multiple nodes which we can visit after visiting a certain node.
- There are multiple tree traversal techniques which decide the order in which the nodes of the tree are to be visited.

Tree Traversal:



In-order Traversal:

- In-order traversal visits the node in the order: Left -> Root -> Right

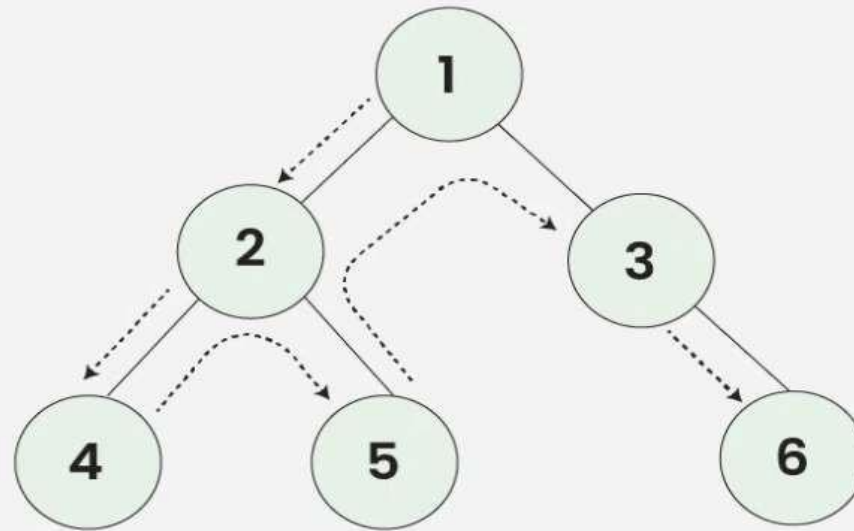


Algorithm for In-order Traversal:

- In-order(tree):
- Traverse the left sub-tree, call In-order(left->sub-tree)
- Visit the root.
- Traverse the right sub-tree, call In-order(right->sub-tree)
- **Uses of In-order Traversal:**
- In the case of binary search trees (BST), In-order traversal gives nodes in non-decreasing order.
- To get nodes of BST in non-increasing order, a variation of In-order traversal where In-order traversal is reversed can be used.
- In-order traversal can be used to evaluate arithmetic expressions stored in expression trees.

Pre-order traversal:

- Preorder traversal visits the node in the order: Root -> Left -> Right



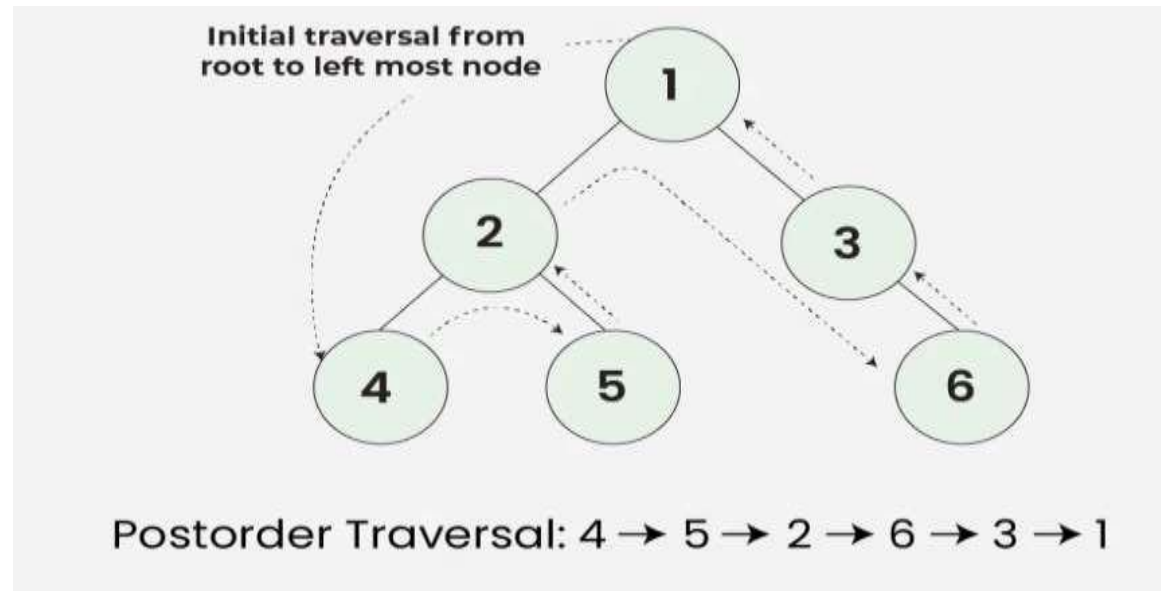
Preorder Traversal: 1 → 2 → 4 → 5 → 3 → 6

Algorithm for Preorder Traversal:

- Preorder(tree):
- Visit the root.
- Traverse the left sub-tree, call Preorder(left->subtree)
- Traverse the right sub-tree, call Preorder(right->subtree)
- **Uses of Preorder Traversal:**
- Preorder traversal is used to create a copy of the tree.
- Preorder traversal is also used to get prefix expressions on an expression tree.

Post-order traversal:

- Post- order traversal visits the node in the order: Left -> Right -> Root

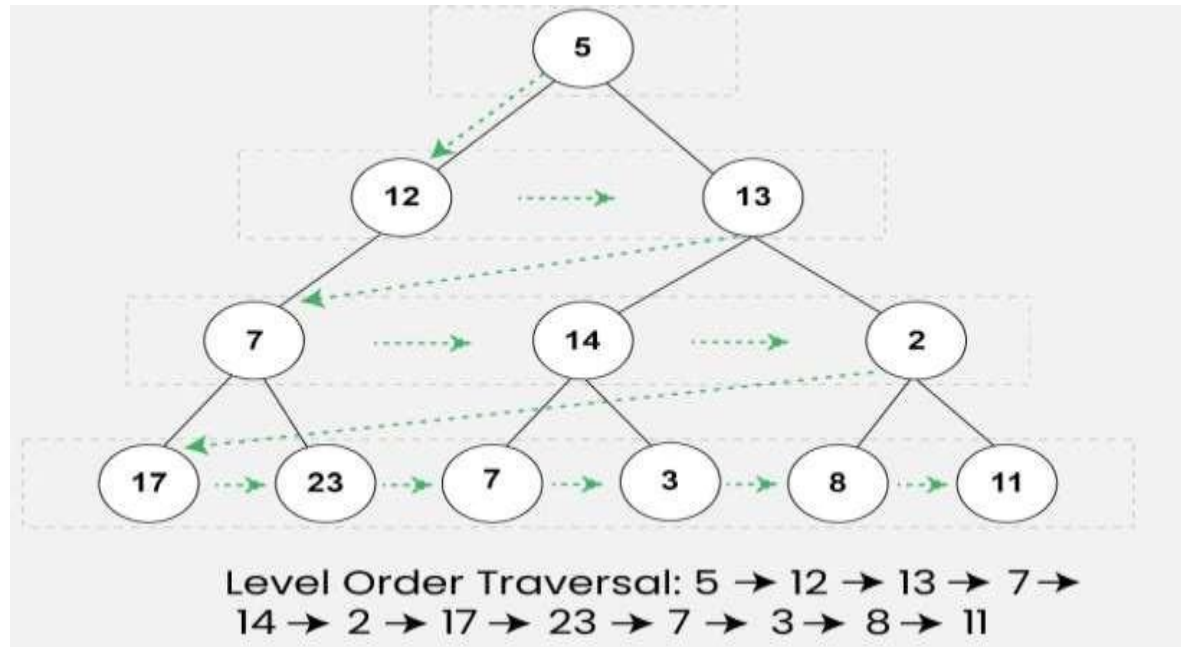


Algorithm for Post-order Traversal:

- **Post-order(tree) :**
- Traverse the left sub-tree, call Post-order(left->sub-tree)
- Traverse the right sub-tree, call Post-order(right->sub-tree)
- Visit the root
- **Uses of Post-order Traversal:**
- Post-order traversal is used to delete the tree. See the question for the deletion of a tree for details.
- Post-order traversal is also useful to get the postfix expression of an expression tree.
- Post-order traversal can help in garbage collection algorithms, particularly in systems where manual memory management is used.

Level Order Traversal :

- Level Order Traversal visits all nodes present in the same level completely before visiting the next level.



Algorithm for Level Order Traversal:

- Create an empty queue Q
- En-queue the root node of the tree to Q
- Loop while Q is not empty
- De-queue a node from Q and visit it
- En-queue the left child of the de-queued node if it exists
- En-queue the right child of the de-queued node if it exists